ТИИСПИС · ВАРИАНТ 26 · «ЯЗЫКИ ПРОГРАММИРОВАНИЯ»

Руководство по ЛР3

Что сделано в третьей лабораторной, где лежит каждый файл, как это работает внутри, как запустить, проверить и показать. Написано для человека, который работу не делал и видит её впервые.

ВСЯ РАБОТА В ЧИСЛАХ

Новых типов вопроса	**5: автор, год, реализация, стандарт, организация**
Правил распознавания вопроса	**40 = 5 типов × 8 языков**
Правил ответа	**5 новых и одно из ЛР2**
Фраз ответа	**10 новых, из них 8 с подстановкой из базы**
Файлов ЛР3 в базе знаний	**60**
Сборка базы	**[1826/1826]**

СОДЕРЖАНИЕ

1. ЛР3 за минуту
2. Задание и чем оно выполнено
3. Что было сделано, по шагам
4. Что уже было до ЛР3
5. Где что лежит: общая карта
6. Папка базы знаний
7. Штатные части NIKA, на которые опирается работа
8. Документы в папке ЛР3
9. Путь вопроса через систему
10. Класс сообщения
11. Общий класс
12. Правило распознавания
13. Правило ответа
14. Правило «сведений нет»
15. Класс фраз и фразы с шаблоном
16. Как из шаблона получается текст
17. Приоритеты: почему проверка данных стоит в правиле
18. Ограничения механизма
19. Что система знает: языки и свойства
20. Какие формулировки она понимает
21. Все фразы ответа
22. Три разбора по журналу
23. Запуск
24. Проверка, что всё на месте
25. Сценарий показа
26. Что открыть в sc-web
27. Если что-то пошло не так
28. Как это проверялось
29. Как поменять ответ системы
30. Откуда взялись файлы и как их пересоздать
31. Приложение А. Все формулировки и ответы
32. Приложение Б. Полный перечень файлов

Пути к файлам базы знаний даны от «своей папки» — `kb/extra/section_subject_domain_of_programming_languages` внутри папки `nika`. Если путь начинается иначе, это сказано рядом.

ЧАСТЬ I · ЧТО ЭТО И ЧТО СДЕЛАНО

1. ЛР3 за минуту

После ЛР2 система понимала, о чём вопрос. После ЛР3 она отвечает на него сведениями из базы знаний.

Суть

В первой лабораторной про каждый язык программирования записаны факты: кто автор, в каком году создан, чем реализован, каким стандартом описан, какая организация сопровождает. В третьей лабораторной система научилась отвечать на вопросы об этих фактах.

ВОПРОС В ЧАТЕ	ЧТО ОТВЕЧАЕТ СИСТЕМА	ПОЧЕМУ ТАК
Какая реализация у Python?	Реализация языка программирования Python — CPython.	у Python в базе записана реализация
Какая реализация у Haskell?	В базе знаний нет таких сведений об этом языке программирования.	у Haskell реализация не записана
Какой компилятор у Python?	Сформулируйте, пожалуйста, по-другому.	такого вопроса система не знает совсем — отвечает одной из штатных фраз непонимания

Первые две строки — главное в работе: **формулировка вопроса одна, а ответ разный**, потому что он зависит от того, что записано в базе.

Три вида файлов, из которых состоит работа

ЧТО	ЗАЧЕМ	СКОЛЬКО
Правило распознавания	по тексту вопроса понять, о чём он и о каком языке	40
Правило ответа	проверить, есть ли в базе нужные сведения, и выбрать набор фраз	5
Фраза	текст ответа с пропусками, в которые подставляются названия из базы	10

Плюс служебное: 5 классов сообщений и 5 классов фраз — «ярлыки», которыми правила пользуются.

2. Задание и чем оно выполнено

Пункты задания взяты из методички дословно. Справа — чем каждый закрыт и где это посмотреть.

Требование → результат

ПУНКТ ЗАДАНИЯ	ЧЕМ ВЫПОЛНЕН	ГДЕ СМОТРЕТЬ
1. Спроектировать логические правила для ответа на сообщения, покрывая каждый класс фраз для ответа	Шесть классов фраз — шесть правил ответа, по одному на класс. Пять правил сделаны в ЛР3, правило об авторе — заранее в ЛР2.	раздел 13, 14, приложение Б
1.1. Правила должны использовать не только классы сообщений, но и выделяемые в сообщении сущности из первой лабораторной работы	В условии пяти правил ответа стоит сущность сообщения — язык из ЛР1 — и проверка, что у неё есть нужное отношение из ЛР1.	раздел 13
1.2. Продемонстрировать различные ответы на одно и то же сообщение в зависимости от состояния базы знаний, используя фразы с шаблонами	Тот же вопрос про Python и про Haskell даёт разные ответы. В ответ подставляются названия из базы. И прямой опыт: в базу добавили строку — на тот же самый вопрос ответ стал другим.	раздел 19, 29
1.3. Ответ на сообщение можно увидеть в пользовательском интерфейсе системы NIKA	Все ответы получены в окне чата. Снимки диалога — в отчёте.	раздел 25, 28
2. Описать фразы ответных сообщений в базе знаний	12 фраз в шести файлах; 10 из них сделаны в ЛР3.	раздел 15, 21
2.1. Фразы сообщений не должны повторяться	12 фраз, все тексты разные.	раздел 21
2.2. Для каждого класса ответных фраз не менее 2 фраз и хотя бы 3 фразы на весь диалог с шаблоном ответа	В каждом из шести классов по две фразы. С шаблоном — 10 из 12.	раздел 21
3. Составить отчёт: цель, задание, вариант, тестирование сценария диалога	Отчёт на 14 страниц с восемью рисунками.	раздел 8
4. Уметь отвечать на вопросы и решать задачи из конца лабораторной	Ответы на восемь вопросов методички и разбор задач — в шпаргалке.	раздел 8
5. Залить изменения в свой форк	**Не сделано.** Нужны адрес форка Вадима, его имя и почта для коммита. Команды готовы.	файл `ЛР3/ИНСТРУКЦИЯ_ЛР3.md`

Что методичка требует к защите

ПУНКТ	ЧЕМ ВЫПОЛНЕН	ГДЕ СМОТРЕТЬ
Протестировать диалог, используя спроектированные правила. Убедиться, что каждое правило может быть применено	В чате заданы все формулировки. Каждое из шести правил ответа сработало — это видно в журнале решателя.	раздел 28
Протестировать правила и фразы в базе знаний системы (навигация по sc-web)	Правила, классы и фразы открываются в sc-web по идентификаторам; снимки этих страниц — в отчёте.	раздел 26

3. Что было сделано, по шагам

В том порядке, в каком это делалось. Файлы первой лабораторной при этом не менялись.

Девять шагов

1. **Выбраны пять типов вопроса**

   По пяти отношениям из ЛР1, у которых значение — отдельная сущность с русским названием: автор, год создания, реализация, стандарт, сопровождающая организация. Такое значение можно подставить в текст ответа.

2. **Описаны классы сообщений**

   Четыре новых класса — по одному на новый тип вопроса (класс для вопроса об авторе был с ЛР2) — и один общий класс для всех пяти. Всего 5 файлов в `messages`.

3. **Сделаны правила распознавания**

   40 файлов: на каждое сочетание «тип вопроса × язык» одно правило с четырьмя формулировками. Из них 32 новых по именам, а 8 про автора: семь заменили одноимённые файлы ЛР2 и один добавлен для языка ассемблера x86.

4. **Сделаны правила ответа**

   Четыре правила второго приоритета — про год, реализацию, стандарт и организацию — и одно правило третьего приоритета на случай, когда сведений нет.

5. **Описаны классы фраз и сами фразы**

   Пять классов фраз. В каждом две фразы: 8 с шаблоном и две без шаблона для ответа «сведений нет».

6. **Собрана база и задан каждый вопрос**

   Сборка без ошибок, 1826 файлов из 1826. В окне чата заданы все формулировки, для которых есть правила. Подробности — раздел 28.

7. **Сняты снимки экрана**

   8 файлов в `ЛР3/Скриншоты`: диалог, экземпляры, правила, класс фраз.

8. **Написаны документы**

   Отчёт, сценарий диалога, шпаргалка к защите, инструкция по переносу. Список — раздел 8.

9. **Собран git-репозиторий для переноса на другой компьютер**

   Готовая папка `nika` с базой внутри и документы всех лабораторных. Скачивание и запуск — раздел 23.

Что изменилось в сделанном раньше

Из файлов ЛР2 тронуты только семь правил распознавания вопроса об авторе. Причина: вопрос об авторе должен вести себя так же, как четыре новых типа.

- Формулировок стало четыре вместо двух: добавились «Кто разработал X?» и «Кто создал X» без знака вопроса.
- В следствии появился второй класс — общий `concept_message_about_language_property`.
- «Кто автор Пролог?» стало «Кто автор Пролога?»: название поставлено в родительный падеж.
- Добавлено восьмое правило — для языка ассемблера x86. У него автор в базе не записан, и на вопрос о нём система теперь отвечает «сведений нет», а не «Сформулируйте по-другому».

Остальное из ЛР2 не менялось: 58 правил «Что такое ...?», 2 правила про декомпозицию, класс сообщения об авторе, правило ответа об авторе и две фразы об авторе.

Что не сделано

- **Коммит и пуш в форк.** Нужны адрес форка Вадима, имя и почта для подписи коммита.
- **Титульный лист отчёта.** В строке «Проверил» стоит фамилия без инициалов — кто принимает работу, знает только Вадим.
- **Wit.ai не используется.** Штатный ключ доступа сервис отвергает, поэтому вопросы распознаются логическими правилами. Это штатный механизм NIKA, и в отчёте он так и описан.

4. Что уже было до ЛР3

Третья лабораторная стоит на двух предыдущих. Без этого абзаца дальнейшее непонятно.

ЛР1 — знания

В базе описаны понятия (язык программирования, компилятор, стандарт языка...), отношения между ними и конкретные экземпляры: 8 языков, их авторы, реализации, стандарты, организации. Каждый экземпляр — отдельный файл в `concepts/instances`.

Вот как выглядит язык Python. Четыре строки, начинающиеся с `nrel_language_author`, `nrel_creation_year`, `nrel_language_implementation` и `nrel_maintaining_organization`, — это факты, о которых спрашивают в ЛР3. Пятой строки, со стандартом (`nrel_language_standard`), здесь нет, и это не ошибка: стандарт для Python в базе не записан.

```
python_language
<- concept_programming_language;
<- concept_interpreted_programming_language;
<- concept_high_level_programming_language;
<- concept_dynamically_typed_programming_language;
<- concept_object_oriented_programming_language;
<- concept_wit_entity;
=> nrel_main_idtf:
    [Python]
    (* <- lang_ru;; *);
    [Python]
    (* <- lang_en;; *);
```

```
=> nrel_idtf:
    [Питон]
    (* <- lang_ru;; *);
=> nrel_note:
    [Python — высокоуровневый язык программирования общего назначения с динамической типизацией и автоматическим
управлением памятью, ориентированный на повышение производительности разработчика и читаемости кода.]
    (* <- lang_ru;; *);
=> nrel_language_author: guido_van_rossum;
=> nrel_creation_year: year_1991;
=> nrel_supported_paradigm: paradigm_object_oriented_programming;
    paradigm_functional_programming;
    paradigm_procedural_programming;
=> nrel_influenced_by: c_language;
=> nrel_language_implementation: cpython_interpreter;
=> nrel_standard_library: python_standard_library;
=> nrel_maintaining_organization: python_software_foundation;
=> nrel_package_manager: pip_package_manager;
=> nrel_ide_support: pycharm_ide;
    visual_studio_code_ide;
-> rrel_reference_implementation: cpython_interpreter;
-> rrel_main_paradigm: paradigm_object_oriented_programming;;
```

ЛР2 — понимание вопроса

Система научилась распознавать три вида вопросов: «Что такое …?», «На что декомпозируется …?» и «Кто создал …?». Распознать — значит повесить на сообщение два ярлыка: **класс** (о чём вопрос) и **сущность** (о ком или о чём).

Там же заранее сделаны правило ответа и две фразы об авторе — без них вопрос «Кто создал Python?» распознавался бы, но остался бы без ответа.

ЧАСТЬ II · ГДЕ ЧТО ЛЕЖИТ

5. Где что лежит: общая карта

Три места: система с базой знаний, документы, и на компьютере, где работа делалась, — ещё инструменты.

В git-репозитории

Всё, что нужно на другом компьютере, лежит в одном репозитории. Как его скачать — раздел 23.

```
nika-lab-v26/
├── README.md                          как скачать и запустить, одной страницей
├── nika/                              система NIKA целиком, база знаний внутри
│   ├── docker-compose.yml             описание четырёх контейнеров
│   ├── docker-compose.override.yml    решателю оставлено одно ядро процессора
│   ├── nika.ini                       настройки решателя
│   └── kb/extra/section_subject_domain_of_programming_languages/    ← своя папка
├── ЛР1/                               документы первой лабораторной
├── ЛР2/                               документы второй лабораторной
├── ЛР3/                               документы третьей лабораторной, это руководство тоже здесь
└── Как_защищать_ЛР2_и_ЛР3.pdf         чем ЛР2 отличается от ЛР3 в общей системе
```

Названные файлы и папки: `README.md`, `nika/docker-compose.yml`, `nika/docker-compose.override.yml`, `nika/nika.ini`, `ЛР1`, `ЛР2`, `ЛР3`, `Как_защищать_ЛР2_и_ЛР3.pdf`.

Тот же набор файлов собран и одним архивом — на случай, если git недоступен.

На компьютере, где работа делалась

Папка `TIISPIS_Vadim`. В ней то же самое и дополнительно:

ПАПКА ИЛИ ФАЙЛ	ЧТО ЭТО
`tools`	скрипты: генераторы правил и фраз, прогон вопросов через чат, сборка отчёта и шпаргалок. В репозиторий не входят.
`repo`	локальная копия git-репозитория; обновляется скриптом `tools/sync_repo.sh`
`bundle/vadim_nika.tar.gz`	то же одним архивом
`docs/superpowers/plans`	план работы и журнал выполнения по шагам
`ЛР2/kb_состояние_ЛР2.zip`	база знаний в том виде, в каком она была после ЛР2, — чтобы сделать в git два отдельных коммита

6. Папка базы знаний

Вся работа трёх лабораторных — в одной папке. В ней 201 файл. К ЛР3 относятся 60.

Что в какой подпапке

```
section_subject_domain_of_programming_languages/
├── section_subject_domain_of_programming_languages.scs    ЛР1: описание предметной области
├── concepts/                    ЛР1: понятия
│   └── instances/               ЛР1: экземпляры — языки, авторы, реализации, стандарты…
├── relations/                   ЛР1: отношения
├── messages/                    классы сообщений           ЛР2: 1   ЛР3: 5
├── logic_rules/
│   ├── classification/          правила распознавания      ЛР2: 60  ЛР3: 40
│   └── answer/                  правила ответа             ЛР2: 1   ЛР3: 5
└── phrases/                     классы фраз                ЛР2: 1   ЛР3: 5
    └── examples/                сами фразы и образцы       ЛР2: 1   ЛР3: 5
```

Регистрировать файлы нигде не нужно: при запуске система собирает всё, что лежит в папке `kb`.

Файлы ЛР3 по видам

ВИД	ГДЕ	СКОЛЬКО	ФОРМАТ	ЧТО ВНУТРИ
Класс сообщения	`messages`	5	`.scs`	название класса и определение; общий класс ещё и перечисляет пять частных
Правило распознавания	`logic_rules/classification`	40	`.gwf`	четыре формулировки вопроса, класс сообщения, общий класс, язык
Правило ответа	`logic_rules/answer`	5	`.gwf`	условие, приоритет, указание на класс фраз
Класс фраз	`phrases`	5	`.scs`	название класса фраз и определение
Фразы	`phrases/examples`	5	`.scs`	две фразы класса и структура-образец, по которой ищутся значения

Имена файлов устроены одинаково, поэтому нужный находится по названию:

ТИП ВОПРОСА	КЛАСС СООБЩЕНИЯ	ОТНОШЕНИЕ ИЗ ЛР1	ПРАВИЛО ОТВЕТА	КЛАСС ФРАЗ
автор	`concept_message_about_author`	`nrel_language_author`	`lr_answer_about_author`	`concept_phrase_about_author`
год создания	`concept_message_about_creation_year`	`nrel_creation_year`	`lr_answer_about_creation_year`	`concept_phrase_about_creation_year`
реализация	`concept_message_about_implementation`	`nrel_language_implementation`	`lr_answer_about_implementation`	`concept_phrase_about_implementation`
стандарт	`concept_message_about_standard`	`nrel_language_standard`	`lr_answer_about_standard`	`concept_phrase_about_standard`
сопровождающая организация	`concept_message_about_maintainer`	`nrel_maintaining_organization`	`lr_answer_about_maintainer`	`concept_phrase_about_maintainer`

Правило распознавания называется `lr_classify_<тип>_of_<язык>_message.gwf`. Тип — `author`, `creation_year`, `implementation`, `standard`, `maintainer`. Язык — `python_lang`, `c_lang`, `cpp_lang`, `java_lang`, `javascript_lang`, `haskell_lang`, `prolog_lang`, `assembly_lang_x86`.

Каждый файл поимённо — в приложении Б.

Два формата

ФОРМАТ	ЧТО ЭТО	ЧЕМ ОТКРЫТЬ
`.scs`	текстовая запись: читается и правится в любом редакторе	любой текстовый редактор
`.gwf`	рисунок графа, сохранённый как XML: узлы, дуги, контуры	редактор KBE; как текст — тоже любой редактор, но читать неудобно

Правила сделаны в `.gwf`, потому что рабочих правил в текстовом виде в базе NIKA нет ни одного — все штатные правила нарисованы. Наши получены из рабочего штатного файла заменой имён и текстов, редактор для этого не нужен.

Фразы с шаблоном, наоборот, записаны текстом. Так же записаны фразы в тестах самой NIKA — например `problem-solver/cxx/dialogControlModule/test/TestsStructures/StandardMessageReply/phrases/phrase1.scs`, по их образцу и сделано. Образец в `.gwf`, собранный без редактора, система генерации фраз не приняла.

7. Штатные части NIKA, на которые опирается работа

Это не наши файлы. Их не меняли, но на защите полезно знать, где они.

В базе знаний

Пути от папки `kb`.

ЧТО	ГДЕ ЛЕЖИТ	ЗАЧЕМ НУЖНО НАМ
Программа обработки сообщения	`section_subject_domain_of_dialogues/section_subject_domain_of_dialog_control/examples/message_processing_program.gwf`	задаёт порядок шагов: распознать, потом ответить
Правило непонимания	`section_subject_domain_of_dialogues/section_subject_domain_of_dialog_control/section_subject_domain_of_dialogue_control_using_template_responses/logic_rules/lr_not_classified_message/lr_not_classified_by_intent_message.gwf`	отвечает, когда вопрос не узнало ни одно правило
Фразы непонимания	`section_subject_domain_of_messages/subject_domain_of_phrases/examples/concept_misunderstanding_phrase/concept_misunderstanding_phrase.gwf`	«Сформулируйте, пожалуйста, по-другому» и подобные
Образец правила распознавания	`section_subject_domain_of_dialogues/section_subject_domain_of_dialog_control/section_subject_domain_of_dialogue_control_using_template_responses/logic_rules/lr_alternative_classification/lr_message_about_entity.gwf`	штатное правило для «Что такое Ника?» — наши правила устроены так же

В решателе

Агенты — программы, которые исполняют правила. Написаны на C++, лежат в `problem-solver/cxx`. Их код в ЛР3 не трогали.

АГЕНТ	ЧТО ДЕЛАЕТ	ГДЕ ИСХОДНИК
`AlternativeMessageTopicClassificationAgent`	перебирает правила распознавания	`problem-solver/cxx/messageClassificationModule/agent/AlternativeMessageTopicClassificationAgent.hpp`
`MessageTopicClassificationAgent`	запасной путь: спрашивает Wit.ai	`problem-solver/cxx/messageClassificationModule/agent/MessageTopicClassificationAgent.hpp`
`StandardMessageReplyAgent`	строит ответ: запускает вывод и сборку фразы	`problem-solver/cxx/dialogControlModule/agent/StandardMessageReplyAgent.hpp`
`DirectInferenceAgent`	перебирает правила ответа по приоритетам	библиотека логического вывода, подключена к решателю
`PhraseGenerationAgent`	подставляет в фразу значения из базы	`problem-solver/cxx/dialogControlModule/agent/PhraseGenerationAgent.hpp`

8. Документы в папке ЛР3

Что сдаётся и что для себя.

Шесть документов и снимки

ФАЙЛ	ДЛЯ ЧЕГО
`ЛР3/ЛР3_Отчет.docx`, **`ЛР3/ЛР3_Отчет.pdf`**	**сдаётся преподавателю.** Цель, задание, вариант, описание правил и фраз, тестирование сценария, выводы. 14 страниц, 8 рисунков.
`ЛР3/Сценарий_диалога.md`	восемь вопросов с ответами, таблица «что записано в базе», все формулировки
`ЛР3/Шпаргалка_к_защите_ЛР3.pdf`, **`ЛР3/Шпаргалка_к_защите_ЛР3.html`**	подготовка к устным вопросам: ответы на восемь вопросов методички и разбор задач. 26 страниц.
`ЛР3/ИНСТРУКЦИЯ_ЛР3.md`	как перенести и проверить, команды для коммита
`ЛР3/Руководство_по_ЛР3.pdf`	этот файл
`ЛР3/Скриншоты`	8 снимков экрана, они же вставлены в отчёт

> ЧЕМ РУКОВОДСТВО ОТЛИЧАЕТСЯ ОТ ШПАРГАЛКИ
>
> Шпаргалка готовит к вопросам преподавателя: теория, определения, задачи. Руководство описывает саму работу: каждый файл, каждый шаг, проверку и запуск.

ЧАСТЬ III · КАК ЭТО РАБОТАЕТ

9. Путь вопроса через систему

Одна и та же цепочка для любого вопроса. Правила ЛР3 работают на втором и четвёртом шаге.

Пять шагов

1. **Чат создаёт сообщение в базе**

   В базе появляется узел-сообщение и ссылка с его текстом. Запускается программа обработки сообщения.

2. **Распознавание — `AlternativeMessageTopicClassificationAgent`**

   Агент перебирает правила распознавания. Правило, у которого текст совпал, добавляет к сообщению класс, общий класс и сущность. **Здесь работают 40 правил ЛР3.**

3. **Если правила не помогли — Wit.ai**

   Сервис недоступен, сообщение помечается как нераспознанное. Для вопросов ЛР3 до этого шага дело не доходит.

4. **Выбор ответа — `DirectInferenceAgent`**

   Агент перебирает правила ответа: сначала первый приоритет, потом второй, потом третий. Останавливается на первом сработавшем. **Здесь работают пять правил ответа ЛР3.** Сработавшее правило называет класс фраз.

5. **Сборка текста — `PhraseGenerationAgent`**

   Агент берёт фразу из названного класса, по её образцу находит в базе значения и подставляет их в пропуски. **Здесь используются фразы ЛР3.** Готовый текст появляется в чате.

То же одной схемой, на примере

```
«Какая реализация у Python?»
        |
        ▼  правило lr_classify_implementation_of_python_lang_message
сообщение ∈ concept_message_about_implementation
сообщение ∈ concept_message_about_language_property
сообщение → rrel_entity → python_language
        |
        ▼  правило lr_answer_about_implementation   (у Python реализация в базе есть)
класс фраз concept_phrase_about_implementation
        |
        ▼  фраза «Реализация языка программирования ${…} — ${…}.»
«Реализация языка программирования Python — CPython.»
```

10. Класс сообщения

Ярлык «о чём вопрос». Сам ничего не делает — им пользуются правила.

Файл целиком

`messages/concept_message_about_implementation.scs`

```
/* Класс сообщений ЛР3: вопрос о свойстве «реализация» языка программирования.
   Сообщение попадает в класс по логическому правилу классификации
   (множество concept_classify_message_rule_priority_1). */

concept_message_about_implementation
<- sc_node_class;
<- concept_intent_possible_class;
=> nrel_main_idtf:
    [сообщение о реализации языка программирования]
    (* <- lang_ru;; *);
    [message about programming language implementation]
    (* <- lang_en;; *);
=> nrel_wit_ai_idtf:
    [about_implementation];
<- rrel_key_sc_element:
    ...
    (*
        => nrel_main_idtf:
            [Опр. (сообщение о реализации языка программирования)]
            (* <- lang_ru;; *);;
        <- definition;;
        <= nrel_sc_text_translation:
            ...
            (*
                -> rrel_example:
                    [Сообщение о реализации языка программирования - атомарное сообщение, содержащее язык
программирования, реализацию которого требуется найти.]
                    (* <- lang_ru;; *);;
            *);;
        <= nrel_using_constants: {
            concept_atomic_message;
            concept_programming_language;
            concept_programming_language_implementation
        };;
    *);;
```

Как это читать

СТРОКА	ЧТО ОЗНАЧАЕТ
`/* … */`	комментарий, система его не читает
`concept_message_about_implementation`	системное имя класса. По нему класс находят правила.
`<- sc_node_class;`	это класс, а не отдельный объект
`<- concept_intent_possible_class;`	«возможный класс цели Wit.ai». Так класс сообщений помечается в шаблоне самой NIKA — `pattern/scs/message.scs`.

СТРОКА	ЧТО ОЗНАЧАЕТ
`=> nrel_main_idtf: […]`	название по-русски и по-английски — оно показывается в sc-web
`=> nrel_wit_ai_idtf: […]`	имя, под которым класс знал бы Wit.ai. Заполнено по тому же шаблону; при распознавании правилами не используется.
`<- rrel_key_sc_element: … definition`	определение класса словами и перечень понятий, через которые оно дано

Четыре класса ЛР3 отличаются только именем, названием и определением. Все четыре относятся к вопросам об одном свойстве языка.

11. Общий класс

Один класс на все пять типов вопроса. Нужен единственному правилу — «сведений нет».

Начало файла

`messages/concept_message_about_language_property.scs`

```
/* Общий класс для всех вопросов о свойстве конкретного языка программирования.
   Правило классификации относит сообщение сразу к частному классу и к этому
   общему: по отношению включения принадлежность сама не переносится, поэтому
   вторая дуга принадлежности создаётся явно.
   По этому классу срабатывает правило ответа третьего приоритета
   lr_answer_about_unknown_language_property — когда нужных сведений в базе нет. */

concept_message_about_language_property
<- sc_node_class;
=> nrel_main_idtf:
    [сообщение о свойстве языка программирования]
    (* <- lang_ru;; *);
    [message about programming language property]
    (* <- lang_en;; *);
<= nrel_inclusion:
    concept_atomic_message;
=> nrel_inclusion:
    concept_message_about_author;
    concept_message_about_creation_year;
    concept_message_about_implementation;
    concept_message_about_standard;
    concept_message_about_maintainer;
```

Строки `=> nrel_inclusion:` перечисляют пять частных классов: общий класс их включает.

Зачем он нужен

Правило «сведений нет» должно срабатывать на любой из пяти типов вопроса. Вместо пяти одинаковых правил сделано одно, и условие у него — «сообщение относится к общему классу».

> **ТОНКОСТЬ, О КОТОРОЙ СПРОСЯТ**
>
> В базе записано, что частный класс включён в общий. Но из этого не следует, что сообщение частного класса система сама считает сообщением общего: принадлежность по включению автоматически не переносится. Поэтому каждое правило распознавания ставит на сообщение оба класса явно.

12. Правило распознавания

Отвечает на вопрос «о чём это сообщение и о каком языке». Пример — `logic_rules/classification/lr_classify_implementation_of_python_lang_message.gwf`.

Что правило говорит

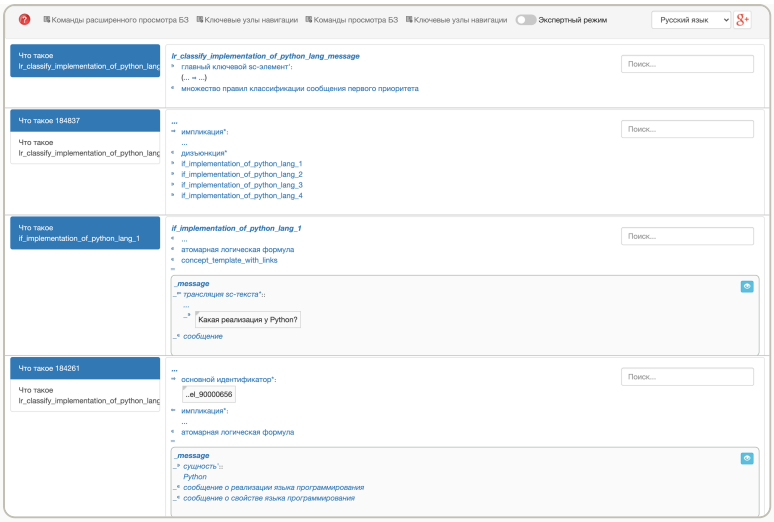
```
ЕСЛИ текст сообщения = «Какая реализация у Python?»
 ИЛИ текст сообщения = «Какая реализация используется для Python?»
 ИЛИ текст сообщения = «Чем реализован Python?»
 ИЛИ текст сообщения = «Какая реализация у Python»
ТО   сообщение ∈ concept_message_about_implementation
     сообщение ∈ concept_message_about_language_property
     сообщение → rrel_entity → python_language
```

Как это устроено в файле

В файле пять контуров — пять обведённых областей графа: четыре условия и одно следствие.

ЧАСТЬ ФАЙЛА	ЧТО В НЕЙ
Контуры `if_implementation_of_python_lang_1` … `if_implementation_of_python_lang_4`	В каждом одно и то же: «есть сообщение, у него есть текстовое представление, в нём ссылка с таким-то текстом». Отличается только текст.
Контур-следствие	Три утверждения о том же сообщении: частный класс, общий класс, сущность с ролью `rrel_entity`.
Кортеж с пометкой `nrel_disjunction`	Собирает четыре условия в «или».
Связь с пометкой `nrel_implication`	Соединяет «или» со следствием: «если … то …».
Узел `lr_classify_implementation_of_python_lang_message`	Само правило. Импликация — его главный ключевой элемент, `rrel_main_key_sc_element`.
Множество `concept_classify_message_rule_priority_1`	В него входит правило. Агент распознавания берёт правила именно отсюда.

В условиях и следствии участвует одна и та же переменная `_message`. Когда условие нашло в базе настоящее сообщение, следствие дорисовывает дуги именно к нему.



Это правило в sc-web: правило, дизъюнкция, четыре условия с текстами, следствие с двумя классами и сущностью.

Почему правил сорок

Текст сравнивается целиком и посимвольно, а сущность в следствии — конкретный узел базы. Правило не умеет вынуть из вопроса слово «Python» и подставить его куда-то. Поэтому на каждое сочетание «тип вопроса × язык» нужно своё правило: 5 × 8 = 40.

Правила сделаны для всех сочетаний, а не только для восьми вопросов сценария. Иначе любой вопрос преподавателя о другом языке дал бы «Сформулируйте по-другому».

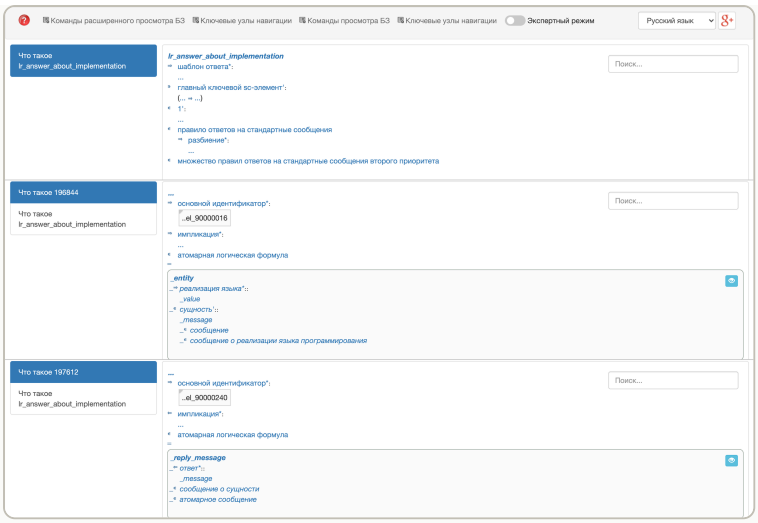
13. Правило ответа

Отвечает на вопрос «каким набором фраз ответить». Пример — `logic_rules/answer/lr_answer_about_implementation.gwf`.

Что правило говорит

```
ЕСЛИ  _message ∈ concept_message                          1. это сообщение
  И   _message ∈ concept_message_about_implementation     2. это вопрос о реализации
  И   _message → rrel_entity → _entity                    3. в нём выделена сущность
  И   _entity → nrel_language_implementation → _value     4. у сущности в базе есть реализация
ТО    _reply_message ∈ concept_message_about_entity
      _reply_message ∈ concept_atomic_message
      (_message → _reply_message) ∈ nrel_reply

приоритет: concept_answer_on_standard_message_rule_priority_2
класс фраз: concept_phrase_about_implementation
```



Правило ответа в sc-web: правило с приоритетом и шаблоном ответа, контур-условие, контур-следствие.

Как это устроено в файле

ЧАСТЬ ФАЙЛА	ЧТО В НЕЙ
Контур-условие	Четыре утверждения сразу. Всё, что лежит в одном контуре, связано «и».
Контур-следствие	Создаётся ответное сообщение `_reply_message` и связывается с вопросом отношением `nrel_reply`. Текста здесь нет.
Связь с пометкой `nrel_implication`	«Если условие, то следствие».
Множество `concept_answer_on_standard_message_rule`	Правило — это правило ответа.
Множество `concept_answer_on_standard_message_rule_priority_2`	Приоритет правила — второй.
Отношение `nrel_answer_pattern`	От правила к кортежу, в котором под номером `rrel_1` стоит класс фраз. Так правило говорит, чем отвечать.

Три вещи, которые надо понимать

1. **Правило не пишет текст.** Оно только создаёт пустое ответное сообщение и называет класс фраз.
2. **Третье условие — это сущность из ЛР1.** Правило работает не с любым сообщением своего класса, а с тем, в котором выделен язык.
3. **Четвёртое условие проверяет, что данные есть.** Переменная `_value` больше нигде не используется — она нужна только как вопрос к базе: «связан ли этот язык хоть с чем-нибудь отношением `nrel_language_implementation`?» Зачем — раздел 17.

Пять правил: что меняется от одного к другому

Меняются три имени: класс сообщения во втором условии, отношение в четвёртом и класс фраз.

ТИП ВОПРОСА	КЛАСС СООБЩЕНИЯ	ОТНОШЕНИЕ ИЗ ЛР1	ПРАВИЛО ОТВЕТА	КЛАСС ФРАЗ
автор	`concept_message_about_author`	`nrel_language_author`	`lr_answer_about_author`	`concept_phrase_about_author`
год создания	`concept_message_about_creation_year`	`nrel_creation_year`	`lr_answer_about_creation_year`	`concept_phrase_about_creation_year`
реализация	`concept_message_about_implementation`	`nrel_language_implementation`	`lr_answer_about_implementation`	`concept_phrase_about_implementation`
стандарт	`concept_message_about_standard`	`nrel_language_standard`	`lr_answer_about_standard`	`concept_phrase_about_standard`
сопровождающая организация	`concept_message_about_maintainer`	`nrel_maintaining_organization`	`lr_answer_about_maintainer`	`concept_phrase_about_maintainer`

Все пять — во втором приоритете. Правило об авторе сделано в ЛР2, остальные четыре — в ЛР3 по его образцу.

14. Правило «сведений нет»

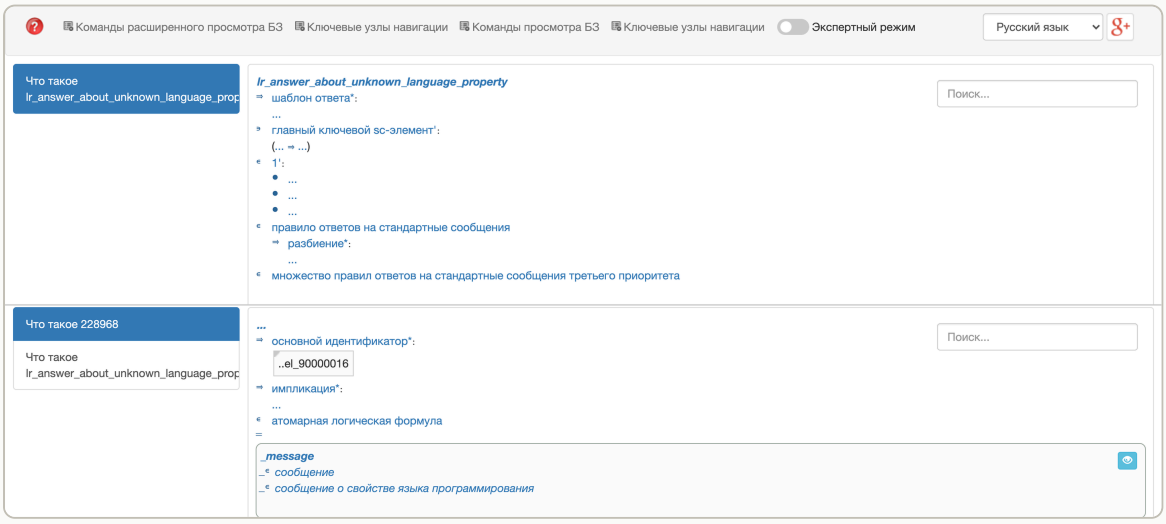
Файл `logic_rules/answer/lr_answer_about_unknown_language_property.gwf`. Одно на все пять типов вопроса.

Что правило говорит

```
ЕСЛИ  _message ∈ concept_message
  И   _message ∈ concept_message_about_language_property     любой вопрос о свойстве языка
ТО    создать ответное сообщение и связать его с вопросом

приоритет: concept_answer_on_standard_message_rule_priority_3
класс фраз: concept_phrase_about_unknown_language_property
```

Условий два. Ни сущность, ни наличие данных не проверяются: правилу достаточно, что вопрос распознан как вопрос о свойстве языка.



Правило третьего приоритета в sc-web: в условии только сообщение и общий класс.

15. Класс фраз и фразы с шаблоном

Правило назвало класс фраз. Теперь из него берётся фраза и заполняется.

Класс фраз

Файл `phrases/concept_phrase_about_implementation.scs` — такой же «ярлык», как класс сообщения: имя, название, определение и строка `<= nrel_inclusion: concept_phrase;` — «это разновидность фразы».

Фразы и образец: файл целиком

`phrases/examples/concept_phrase_about_implementation.scs`

```
/* Фразы ответа класса concept_phrase_about_implementation (ЛР3).

   В тексте фразы ${...} — место подстановки; имя в скобках совпадает
   с nrel_system_identifier переменной-ссылки. Подстановок в каждой фразе две:
   с тремя агент генерации фраз фразу не собирает.
   Обе фразы класса используют один образец @tpl_impl. */

@var_language_name_impl = _[];;
@var_language_name_impl
=> nrel_system_identifier:
         [_language_name_impl];;

@var_impl_name = _[];;
@var_impl_name
=> nrel_system_identifier:
         [_impl_name];;

/* Образец: сообщение, выделенный в нём язык программирования, его русский
   идентификатор, значение отношения nrel_language_implementation и русский идентификатор
   этого значения. Если у языка такого отношения нет, образец не находится. */
@tpl_impl = [*
         concept_message
         _-> _message;;

         _message
         _-> rrel_entity:: _language_impl;;

         _language_impl
         _=> nrel_main_idtf::
                  @var_language_name_impl
                  (* _<- lang_ru;; *);;

         _language_impl
         _=> nrel_language_implementation:: _implementation;;

         _implementation
         _=> nrel_main_idtf::
                  @var_impl_name
                  (* _<- lang_ru;; *);;
*];;

concept_phrase_about_implementation ->
         [Реализация языка программирования ${_language_name_impl} — ${_impl_name}.]
         (*
```

```
                <- lang_ru;;
                => nrel_phrase_template: @tpl_impl;;
        *);;

concept_phrase_about_implementation ->
        [Для языка программирования ${_language_name_impl} используется реализация «${_impl_name}».]
        (*
                <- lang_ru;;
                => nrel_phrase_template: @tpl_impl;;
        *);;
```

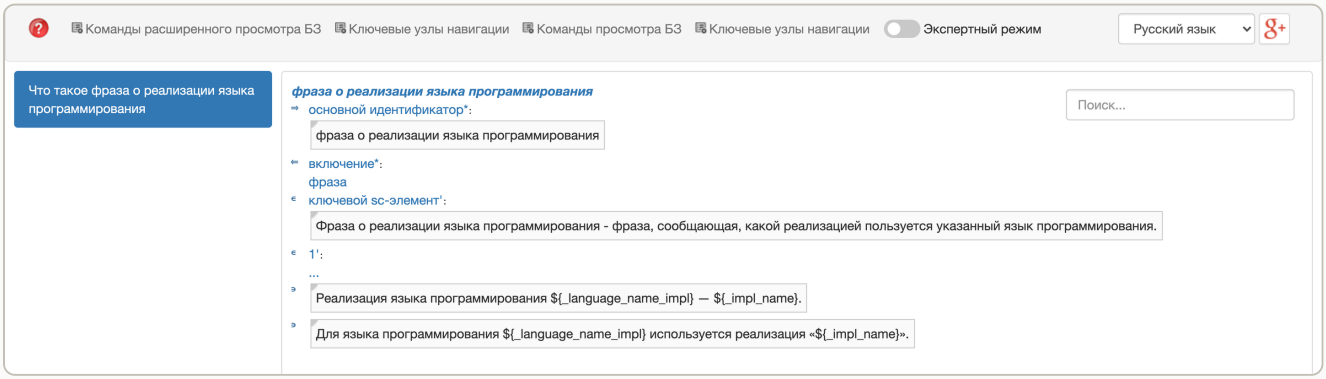
Как это читать: три части

ЧАСТЬ	СТРОКИ	ЧТО ОЗНАЧАЕТ
Две переменные	`@var_language_name_impl = _[];;` `@var_impl_name = _[];;`	Два «пропуска». Запись `_[]` — переменная-ссылка, то есть «какой-то текст, пока неизвестно какой». Строка `nrel_system_identifier` даёт пропуску имя: `_language_name_impl` и `_impl_name`.
Структура-образец	`@tpl_impl = [* … *];;`	Маленький рисунок графа из переменных: что именно искать в базе, чтобы заполнить пропуски.
Две фразы	`concept_phrase_about_implementation -> […]`	Тексты с метками `${имя}`. Строка `nrel_phrase_template` привязывает к фразе образец.

Образец по строкам

СТРОКА ОБРАЗЦА	ЧТО ИЩЕТСЯ В БАЗЕ
`concept_message _-> _message;;`	сообщение, на которое отвечаем
`_message _-> rrel_entity:: _language_impl;;`	его сущность — язык
`_language_impl _=> nrel_main_idtf:: @var_language_name_impl (* _<- lang_ru;; *);;`	русское название языка → первый пропуск
`_language_impl _=> nrel_language_implementation:: _implementation;;`	то, с чем язык связан отношением «реализация языка*»
`_implementation _=> nrel_main_idtf:: @var_impl_name (* _<- lang_ru;; *);;`	русское название реализации → второй пропуск

Значки: `_->` и `_=>` — дуга-переменная («какая-то дуга такого вида»); двойное двоеточие после имени отношения — то же самое про дугу самого отношения. Всё, что начинается с подчёркивания, — переменная.



Класс фраз в sc-web: две фразы и общая для них структура-образец.

16. Как из шаблона получается текст

На вопросе «Какая реализация у Python?»: откуда берётся каждое слово ответа.

По шагам

1. **Сообщение и его сущность**

   Правило распознавания уже связало сообщение с узлом `python_language`. Образец подставляет его на место `_language_impl`.

2. **Первый пропуск — название языка**

   В файле `concepts/instances/languages/python_language.scs` у узла есть русский основной идентификатор:

   ```
   => nrel_main_idtf:
       [Python]
       (* <- lang_ru;; *);
   ```

   Текст «Python» попадает в `${_language_name_impl}`.

3. **Реализация языка**

   В том же файле:

   ```
   => nrel_language_implementation: cpython_interpreter;
   ```

   Узел `cpython_interpreter` встаёт на место `_implementation`.

4. **Второй пропуск — название реализации**

   В файле `concepts/instances/implementations/cpython_interpreter.scs`:

   ```
   => nrel_main_idtf:
       [CPython]
       (* <- lang_ru;; *);
   ```

   Текст «CPython» попадает в `${_impl_name}`.

5. **Готовый ответ**

   «Реализация языка программирования Python — CPython.» или вторая фраза класса: «Для языка программирования Python используется реализация «CPython».» Какую из двух взять, система решает случайно.

> **ОТСЮДА ГЛАВНОЕ СВОЙСТВО**
>
> В тексте фразы нет ни слова «Python», ни слова «CPython». Оба взяты из базы в момент ответа. Изменится запись в базе — изменится и ответ, хотя ни правило, ни фразу никто не трогал.

17. Приоритеты: почему проверка данных стоит в правиле

Самое важное техническое решение работы. Его стоит уметь объяснить своими словами.

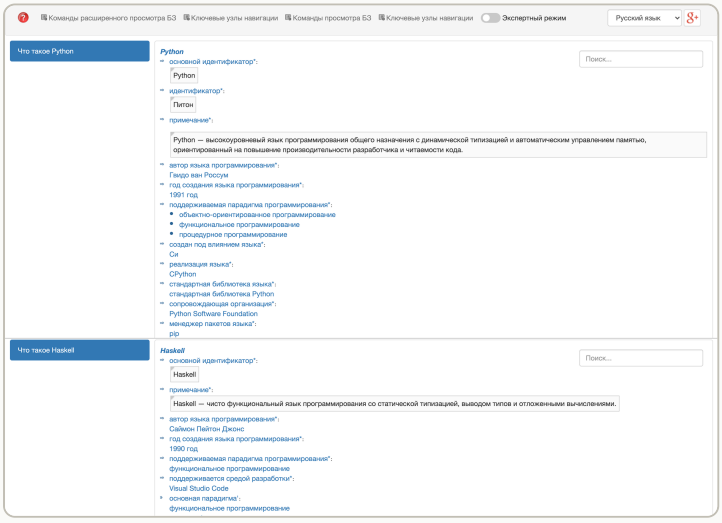
Как идёт перебор

Правила ответа разложены по трём множествам-приоритетам. Агент вывода проверяет их по порядку и **останавливается на первом сработавшем правиле** — в журнале это строка `Target is achieved`.

ПРИОРИТЕТ	ЧТО ТАМ ЛЕЖИТ
1	штатные правила NIKA: приветствие, непонимание, декомпозиция и другие
2	пять наших правил по существу и штатное правило «Что такое ...?»
3	наше правило «сведений нет» и штатные правила того же рода

Два вопроса — два пути

	КАКАЯ РЕАЛИЗАЦИЯ У PYTHON?	КАКАЯ РЕАЛИЗАЦИЯ У HASKELL?
Распознавание	вопрос о реализации, язык Python	вопрос о реализации, язык Haskell
Правило второго приоритета	**сработало: реализация записана**	**не сработало: четвёртое условие ложно**
Правило третьего приоритета	до него не дошло	**сработало**
Ответ	по существу	«сведений нет»



Экземпляры в sc-web: у Python есть «реализация языка*», у Haskell её нет.

Что было бы без четвёртого условия

Естественно ожидать так: «у Haskell образец фразы не найдётся, фраза не соберётся, и система сама перейдёт к следующему правилу». **Это не так.**

Правило второго приоритета без проверки данных сработало бы и для Haskell. Перебор на этом остановился бы. Фраза с шаблоном не собралась бы, потому что образцу нечего найти. И ответа не было бы вообще — до правила третьего приоритета вывод уже не дошёл бы.

> **СКАЗАТЬ ВСЛУХ**
>
> Проверка наличия данных стоит в самом правиле, потому что вывод останавливается на первом сработавшем правиле. Если данных нет, правило второго приоритета не срабатывает, и отвечает правило третьего.

18. Ограничения механизма

То, что система не умеет. Лучше назвать самому, чем дать найти.

Четыре ограничения

ОГРАНИЧЕНИЕ	КАК ПРОЯВЛЯЕТСЯ	ЧТО С ЭТИМ СДЕЛАНО
Текст вопроса сравнивается посимвольно	«какая реализация у python?» с маленькой буквы не распознаётся	на каждое сочетание по четыре формулировки, одна из них без знака вопроса
Правило не обобщает	вопрос о языке, которого нет в базе, не распознаётся	правила сделаны для всех восьми языков и всех пяти типов
В одной фразе не больше двух подстановок	с тремя метками `${…}` фраза не собирается, ответа нет	в каждой фразе ровно две: название языка и значение
Имена переменных общие на всю базу	два файла с одинаковым именем переменной мешают друг другу	у каждого класса фраз свои имена: `_language_name_impl`, `_language_name_std` и так далее

ЧАСТЬ IV · ЧТО ОТВЕЧАЕТ СИСТЕМА

19. Что система знает: языки и свойства

Таблица построена по файлам экземпляров. Значение — ответ по существу, прочерк — ответ «сведений нет».

Языки и их свойства

ЯЗЫК	АВТОР	ГОД СОЗДАНИЯ	РЕАЛИЗАЦИЯ	СТАНДАРТ	СОПРОВОЖДАЮЩАЯ ОРГАНИЗАЦИЯ
Python	Гвидо ван Россум	1991 год	CPython	—	Python Software Foundation
Си	Деннис Ритчи	1972 год	GCC	ISO/IEC 9899:2018	Международная организация по стандартизации

ЯЗЫК	АВТОР	ГОД СОЗДАНИЯ	РЕАЛИЗАЦИЯ	СТАНДАРТ	СОПРОВОЖДАЮЩАЯ ОРГАНИЗАЦИЯ
Си++	Бьёрн Страуструп	1985 год	GCC	ISO/IEC 14882:2020	Международная организация по стандартизации
Java	Джеймс Гослинг	1995 год	виртуальная машина Java	—	Oracle Corporation
JavaScript	Брендан Эйх	1995 год	—	ECMA-262	—
Haskell	Саймон Пейтон Джонс	1990 год	—	—	—
Пролог	Ален Колмероэ	1972 год	—	—	—
язык ассемблера x86	—	—	—	—	—

Из 40 сочетаний 25 дают ответ по существу и 15 — фразу «сведений нет». Пробелы настоящие: так база была заполнена в первой лабораторной, под ЛР3 её не подгоняли.

20. Какие формулировки она понимает

По четыре на каждое сочетание, всего 160. Тексты взяты из файлов правил.

Формулировки по типам

ТИП ВОПРОСА	ЧЕТЫРЕ ФОРМУЛИРОВКИ (НА ПРИМЕРЕ PYTHON)
автор	Кто создал Python? Кто автор Python? Кто разработал Python? Кто создал Python
год создания	В каком году создан Python? Когда создан Python? Когда появился Python? В каком году создан Python
реализация	Какая реализация у Python? Какая реализация используется для Python? Чем реализован Python? Какая реализация у Python

ТИП ВОПРОСА	ЧЕТЫРЕ ФОРМУЛИРОВКИ (НА ПРИМЕРЕ PYTHON)
стандарт	Какой стандарт у Python? Каким стандартом описан Python? Какой стандарт действует для Python? Какой стандарт у Python
сопровождающая организация	Кто сопровождает Python? Какая организация сопровождает Python? Кто поддерживает Python? Кто сопровождает Python

Для другого языка вместо «Python» ставится его русское название из базы: Си, Си++, Java, JavaScript, Haskell, Пролог, язык ассемблера x86.

> **ДВА НАЗВАНИЯ СКЛОНЯЮТСЯ**
>
> Там, где название стоит после «у», «для» или «автор», оно в родительном падеже: «Какой стандарт у **Пролога**?», «Кто автор **языка ассемблера x86**?». В формулировках с именительным падежом — как есть: «Кто создал Пролог?». Точный текст каждого вопроса — в приложении А.

Регистр и знак вопроса значимы. Вопрос надо набирать с большой буквы и ровно так, как в таблице.

21. Все фразы ответа

Шесть классов, в каждом по две фразы. Тексты взяты из файлов в `phrases/examples`.

Фразы по классам

КЛАСС ФРАЗ	ТЕКСТ ФРАЗЫ	С ШАБЛОНОМ	СДЕЛАНА В
`concept_phrase_about_author`	Автор языка программирования ${_language_name} — ${_author_name}.	да	ЛР2
`concept_phrase_about_author`	Язык программирования ${_language_name} создал ${_author_name}.	да	ЛР2
`concept_phrase_about_creation_year`	Год создания языка программирования ${_language_name_year} — ${_year_name}.	да	ЛР3
`concept_phrase_about_creation_year`	${_language_name_year}: год создания — ${_year_name}.	да	ЛР3
`concept_phrase_about_implementation`	Реализация языка программирования ${_language_name_impl} — ${_impl_name}.	да	ЛР3
`concept_phrase_about_implementation`	Для языка программирования ${_language_name_impl} используется реализация «${_impl_name}».	да	ЛР3
`concept_phrase_about_maintainer`	Язык программирования ${_language_name_org} сопровождает ${_org_name}.	да	ЛР3
`concept_phrase_about_maintainer`	Сопровождением языка программирования ${_language_name_org} занимается ${_org_name}.	да	ЛР3
`concept_phrase_about_standard`	Стандарт языка программирования ${_language_name_std} — ${_std_name}.	да	ЛР3

КЛАСС ФРАЗ	ТЕКСТ ФРАЗЫ	С ШАБЛОНОМ	СДЕЛАНА В
`concept_phrase_about_standard`	Язык программирования ${_language_name_std} описан стандартом ${_std_name}.	да	ЛР3
`concept_phrase_about_unknown_language_property`	В базе знаний нет таких сведений об этом языке программирования.	нет	ЛР3
`concept_phrase_about_unknown_language_property`	Этого о данном языке программирования я пока не знаю.	нет	ЛР3

С шаблоном 10 фраз из 12; в ЛР3 сделано 10, из них с шаблоном 8. Тексты не повторяются.

Система берёт любую из двух фраз класса случайно. Поэтому на один и тот же вопрос формулировка ответа бывает то одной, то другой — это не ошибка.

22. Три разбора по журналу

Строки взяты из журнала решателя во время проверочного прогона. Журнал открывается командой `docker compose logs -f problem-solver` из папки `nika`.

Как читать выдержки

СТРОКА ЖУРНАЛА	ЧТО ОЗНАЧАЕТ
`There is N formulas in K set`	в приоритете K лежит N правил, начинается их перебор
`Compute atomic logical formula if_… true`	условие с таким именем выполнилось — текст совпал
`Message is classified.`	сообщение распознано
`Trying to generate by formula: …`	проверяется правило ответа с таким именем
`Compute atomic logical formula true`	условие только что названного правила выполнилось. Имени нет, потому что контуры в правилах ответа безымянные.
`Target is achieved`	правило сработало, перебор остановлен
`MessageHandler: the logic rule … is found`	какое правило дало ответ
`MessageHandler: the phrase with the content … is found`	какая фраза взята
`Generated text: …`	готовый текст ответа

Многоточие в выдержке заменяет штатные правила NIKA, которые были проверены и не подошли. Сначала в журнале идёт перебор всех 113 правил распознавания: 100 наших и 13 штатных.

Порядок правил внутри одного приоритета система выбирает сама, в вашем журнале он может быть другим. Порядок самих приоритетов постоянный.

Разбор 1. «Какая реализация у Python?» — данные есть

```
There is 113 formulas in 1 set
Compute atomic logical formula if_implementation_of_python_lang_1 true
Message is classified.
There is 10 formulas in 1 set
```

```
…
There is 7 formulas in 2 set
Trying to generate by formula: lr_answer_about_maintainer
Trying to generate by formula: lr_answer_about_standard
Trying to generate by formula: lr_answer_about_implementation
Compute atomic logical formula  true
Target is achieved
MessageHandler: the logic rule lr_answer_about_implementation is found
MessageHandler: the phrase with the content "Реализация языка программирования ${_language_name_impl} —
${_impl_name}." is found
Generated text: "Реализация языка программирования Python — CPython."
```

Распознавание: выполнилось условие `if_implementation_of_python_lang_1`. Выбор ответа: первый приоритет пройден без результата, во втором сработало `lr_answer_about_implementation`. Дальше взята фраза и собран текст.

Разбор 2. «Какая реализация у Haskell?» — данных нет

```
There is 113 formulas in 1 set
Compute atomic logical formula if_implementation_of_haskell_lang_1 true
Message is classified.
There is 10 formulas in 1 set
…
There is 7 formulas in 2 set
Trying to generate by formula: lr_answer_about_maintainer
Trying to generate by formula: lr_answer_about_standard
Trying to generate by formula: lr_answer_about_implementation
…
Trying to generate by formula: lr_answer_about_creation_year
Trying to generate by formula: lr_answer_about_author
There is 5 formulas in 3 set
…
Trying to generate by formula: lr_answer_about_unknown_language_property
Compute atomic logical formula  true
Target is achieved
MessageHandler: the logic rule lr_answer_about_unknown_language_property is found
MessageHandler: the phrase with the content "Этого о данном языке программирования я пока не знаю." is found
Generated text: "Этого о данном языке программирования я пока не знаю."
```

Распознавание такое же. Во втором приоритете проверены все правила, и ни после одного нет строки `Compute atomic logical formula true`: у правила про реализацию ложно четвёртое условие, остальные не подходят по классу сообщения. В третьем приоритете сработало `lr_answer_about_unknown_language_property`.

Разбор 3. «Какой компилятор у Python?» — вопрос без правила

```
There is 113 formulas in 1 set
Message is not classified.
MessageTopicClassificationAgent started
[Info]: {"code":"no-auth","error":"Invalid user: FB log-in user is no longer supported"}
[Warning]: MessageTopicClassifier: Message intent class is not found.
[Warning]: MessageTopicClassifier: Message trait class is not found.
[Warning]: MessageTopicClassifier: Message entities are not found.
```

```
There is 10 formulas in 1 set
…
Trying to generate by formula: lr_not_classified_by_intent_message
Compute atomic logical formula  true
Target is achieved
MessageHandler: the logic rule lr_not_classified_by_intent_message is found
MessageHandler: the phrase with the content "Я не понимаю Вас" is found
Generated text: "Я не понимаю Вас"
```

Ни одно правило распознавания не подошло. Запустился запасной путь через Wit.ai, сервис ответил отказом. Сообщение помечено как нераспознанное, и в первом приоритете сработало штатное правило `lr_not_classified_by_intent_message`.

> ЗАЧЕМ ЭТО ЗНАТЬ
>
> Строка `no-auth` в журнале — не поломка. Она появляется каждый раз, когда вопрос не узнало ни одно правило.

Чем «сведений нет» отличается от «непонимания»

	«СВЕДЕНИЙ НЕТ»	НЕПОНИМАНИЕ
Пример	Какая реализация у Haskell?	Какой компилятор у Python?
Распознан ли вопрос	**да: известны тип вопроса и язык**	**нет**
Какое правило отвечает	наше, третий приоритет	штатное, первый приоритет
Чьи фразы	наши, из ЛР3	штатные

ЧАСТЬ V · ЗАПУСК, ПРОВЕРКА, ПОКАЗ

23. Запуск

Нужны git и установленный, запущенный Docker.

Скачать

```
git clone https://github.com/enterpraizer/nika-lab-v26.git
cd nika-lab-v26
```

Позже, чтобы забрать правки или следующую лабораторную, в этой же папке выполнить `git pull` и перезапустить систему.

Запустить

```
cd nika
docker compose up -d --no-build
```

В первый раз скачиваются образы. Потом минуту-две собирается база знаний. Когда команда завершилась, открыть в браузере:

АДРЕС	ЧТО ТАМ
`http://localhost:3033`	окно чата — сюда задаются вопросы
`http://localhost:8000`	sc-web — просмотр базы знаний

> **ЕСЛИ НА ЭТОМ КОМПЬЮТЕРЕ УЖЕ РАБОТАЕТ ДРУГАЯ NIKA**
>
> Адреса у всех копий одни и те же. Сначала остановить ту: `docker compose stop` в её папке.

Остановить и запустить снова

```
docker compose stop
docker compose start
```

При каждом запуске база знаний собирается заново из файлов. История чата при этом пропадает — это нормально.

Перезапустить, если Ника замолчала

```
docker compose restart problem-solver
docker compose restart py-sc-server
```

Обе команды и именно в таком порядке. Вторая обязательна: без неё Ника бесконечно «печатает» и не отвечает.

Как задавать вопросы

- Отправлять **кнопкой с самолётиком**. Клавиша Enter не отправляет.
- Большая буква в начале, знак вопроса в конце — текст сравнивается посимвольно.
- Следующий вопрос — после ответа на предыдущий.
- Красная панель с надписью `TypeError` при первом открытии чата закрывается кнопкой «Close». Это ошибка самой NIKA на пустом чате, на работу не влияет.

24. Проверка, что всё на месте

Три проверки за минуту: база собралась, файлы на месте, система отвечает.

1. База собралась полностью

```
docker compose logs problem-solver | grep -oE "\[[0-9]+/[0-9]+\]" | tail -1
```

Должно напечатать `[1826/1826]` — два одинаковых числа. Если первое меньше — сборка ещё идёт. Если числа другие — в папку базы что-то добавлено или из неё что-то пропало.

2. Файлы на месте

```
ls kb/extra/section_subject_domain_of_programming_languages/logic_rules/classification | wc -l
ls kb/extra/section_subject_domain_of_programming_languages/logic_rules/answer | wc -l
ls kb/extra/section_subject_domain_of_programming_languages/phrases/examples | wc -l
```

Должно получиться 100, 6 и 6.

3. Система отвечает

В чате отправить пару:

```
Какая реализация у Python?
Какая реализация у Haskell?
```

На первый вопрос должен прийти ответ с «CPython», на второй — одна из двух фраз «сведений нет».

25. Сценарий показа

Восемь вопросов из отчёта. Вопросы 1–2, 3–4 и 5–6 — пары: формулировка одна, язык разный.

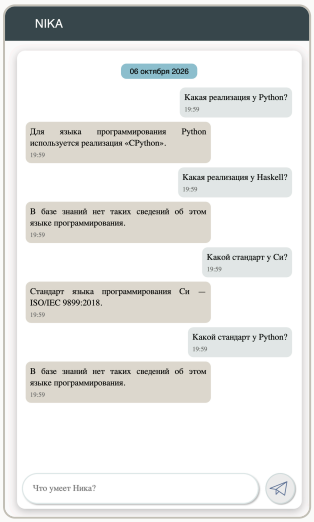
Что набирать

```
Какая реализация у Python?
Какая реализация у Haskell?
Какой стандарт у Си?
Какой стандарт у Python?
Кто сопровождает Java?
Кто сопровождает JavaScript?
В каком году создан Си++?
Кто создал Haskell?
```

Что ответила система при проверке

№	ВОПРОС	ЧТО ОТВЕТИЛА СИСТЕМА	СУЩНОСТЬ	КАКОЕ ПРАВИЛО ОТВЕТИЛО
1	Какая реализация у Python?	Реализация языка программирования Python — CPython.	`python_language`	`lr_answer_about_implementation`, приоритет 2
2	Какая реализация у Haskell?	Этого о данном языке программирования я пока не знаю.	`haskell_language`	`lr_answer_about_unknown_language_property`, приоритет 3
3	Какой стандарт у Си?	Стандарт языка программирования Си — ISO/IEC 9899:2018.	`c_language`	`lr_answer_about_standard`, приоритет 2
4	Какой стандарт у Python?	Этого о данном языке программирования я пока не знаю.	`python_language`	`lr_answer_about_unknown_language_property`, приоритет 3
5	Кто сопровождает Java?	Язык программирования Java сопровождает Oracle Corporation.	`java_language`	`lr_answer_about_maintainer`, приоритет 2
6	Кто сопровождает JavaScript?	Этого о данном языке программирования я пока не знаю.	`javascript_language`	`lr_answer_about_unknown_language_property`, приоритет 3
7	В каком году создан Си++?	Год создания языка программирования Си++ — 1985 год.	`cpp_language`	`lr_answer_about_creation_year`, приоритет 2
8	Кто создал Haskell?	Язык программирования Haskell создал Саймон Пейтон Джонс.	`haskell_language`	`lr_answer_about_author`, приоритет 2

В каждом классе две фразы, поэтому при показе формулировка ответа может оказаться второй из двух. Оба варианта для каждого вопроса — в приложении А.



Начало сценария в окне чата — рисунок из отчёта.

Что говорить по ходу

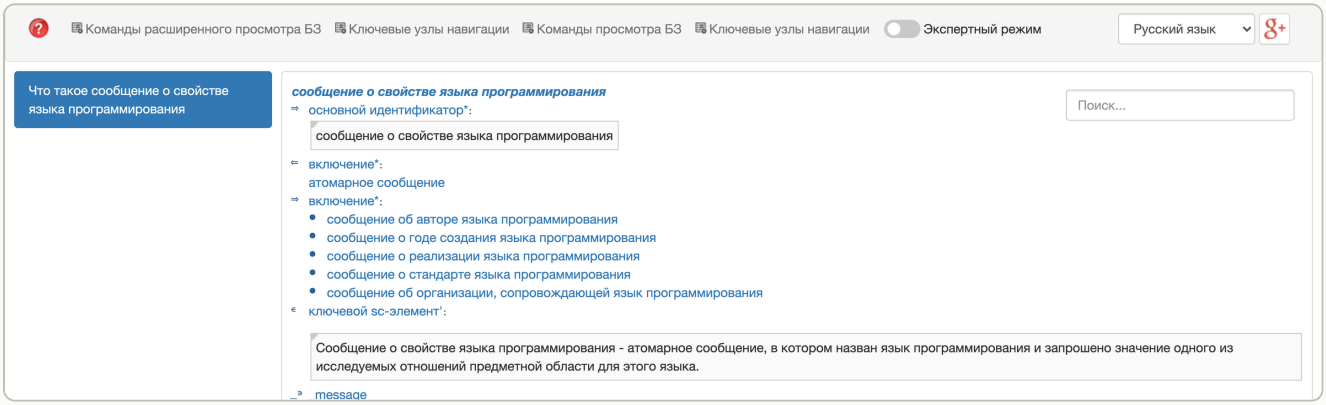
1. После первых двух вопросов: «Формулировка одна, ответы разные — у Python реализация в базе записана, у Haskell нет».
2. После пар про стандарт и организацию: «То же самое для других отношений — каждое правило проверяет своё».
3. После седьмого и восьмого: «В ответ подставлены название языка и значение из базы. В тексте фразы их нет».

26. Что открыть в sc-web

Адрес — `http://localhost:8000/?sys_id=` и идентификатор. Или ввести идентификатор в строку поиска.

Шесть страниц в порядке показа

ИДЕНТИФИКАТОР	ЧТО ВИДНО	ЧТО СКАЗАТЬ
`python_language`, затем `haskell_language`	у первого есть строка «реализация языка*», у второго нет	«Это состояние базы, от которого зависит ответ».
`lr_classify_implementation_of_python_lang_message`	четыре условия с текстами и следствие	«Правило распознавания: четыре формулировки, в следствии два класса и сущность».
`lr_answer_about_implementation`	условие из четырёх утверждений, приоритет, шаблон ответа	«Правило ответа: класс сообщения, сущность и проверка, что у сущности есть реализация».
`concept_phrase_about_implementation`	две фразы и структура-образец	«Фразы с пропусками. По образцу из базы берутся два названия».
`lr_answer_about_unknown_language_property`	условие из двух утверждений, третий приоритет	«Отвечает, когда правило второго приоритета не сработало».
`concept_message_about_language_property`	общий класс и пять частных	«По этому классу срабатывает правило „сведений нет“».



Общий класс сообщений в sc-web: включает пять частных классов.

27. Если что-то пошло не так

Признак, причина, действие.

Шесть случаев

ЧТО ВИДНО	ПРИЧИНА	ЧТО ДЕЛАТЬ
Ника бесконечно «печатает»	после перезапуска решателя не перезапущен контейнер `py-sc-server`	`docker compose restart py-sc-server`
На вопрос нет ответа секунд десять, хотя до этого отвечала	решатель изредка теряет уже готовый ответ — недостаток самой NIKA	отправить тот же вопрос ещё раз
Перестали открываться и чат, и sc-web	решатель завис — тоже недостаток самой NIKA	две команды перезапуска из раздела 23

ЧТО ВИДНО	ПРИЧИНА	ЧТО ДЕЛАТЬ
«Сформулируйте, пожалуйста, по-другому» на вопрос из сценария	текст набран не так: маленькая буква, лишний пробел, другой падеж	набрать точно по приложению А
Красная панель `TypeError` поверх чата	ошибка NIKA на пустом чате	закрыть кнопкой «Close»
Чат вообще не открывается	контейнеры не запущены или порт занят другой копией NIKA	`docker compose ps`; остановить другую копию

Про зависание: в файле `docker-compose.override.yml` решателю оставлено одно ядро процессора. С этой настройкой он зависает реже, но совсем зависания не пропадают.

28. Как это проверялось

Не «должно работать», а что именно было запущено и что получено.

Проверочный прогон 8 октября 2026

ЧТО ПРОВЕРЕНО	РЕЗУЛЬТАТ
Сборка базы знаний	1826 файлов из 1826, без ошибок
Вопросы в настоящем окне чата	заданы все 280 формулировок, для которых есть правила (160 из ЛР3 и 120 из ЛР2), и один вопрос без правила — всего 281
Ожидаемый ответ получен	на 281 из 281
С первой отправки	275; ещё 6 — со второй, после того как завис и был перезапущен решатель
Как шёл прогон	двумя заходами подряд на одной и той же базе: сначала 231 вопрос, затем оставшиеся 50 — первый заход оборвался из-за сбоя браузера, которым задавались вопросы
Сценарий из восьми вопросов	пройден, ответы — в разделе 25
Каждое из шести правил ответа	сработало хотя бы раз — видно по строкам журнала
Изменение базы меняет ответ	проверено опытом — раздел 29

«Ожидаемый ответ» для вопроса по существу — текст, в котором стоит значение из базы; для вопроса без данных — одна из двух фраз «сведений нет»; для вопроса без правила — штатная фраза непонимания.

Чего проверка не покрывает

- Прогон сделан на macOS. На Linux полный прогон не делался.
- Запуск из свежего клона репозитория проверен отдельно: база собралась полностью, на контрольные вопросы система ответила.

- Зависания решателя и потерянные ответы — недостатки самой NIKA. Они не устранены, для них есть порядок действий в разделе 27.

29. Как поменять ответ системы

Три опыта. Каждый выполнен на копии базы знаний, ответы записаны из чата. В рабочую базу эти правки не вносились.

Что получилось

ВОПРОС	ОТВЕТ ДО ПРАВКИ	ОТВЕТ ПОСЛЕ ПРАВКИ
Кто сопровождает Пролог?	В базе знаний нет таких сведений об этом языке программирования.	Сопровождением языка программирования Пролог занимается Международная организация по стандартизации.
Какой стандарт у Пролога?	Этого о данном языке программирования я пока не знаю.	Стандарт языка программирования Пролог — ISO/IEC 13211-1:1995.
Какая реализация у Python?	Реализация языка программирования Python — CPython.	Этого о данном языке программирования я пока не знаю.

Во всех трёх случаях не менялись ни правила, ни фразы. Менялись только файлы экземпляров из первой лабораторной.

> **ЭТО И ЕСТЬ ПУНКТ 1.2 ЗАДАНИЯ**
> Одно и то же сообщение, разные ответы в зависимости от состояния базы знаний. Если преподаватель попросит показать это вживую — любой из трёх опытов занимает пять минут.

Опыт 1. Добавить строку — появится ответ по существу

В файл `concepts/instances/languages/prolog_language.scs` перед последней строкой добавить:

```
=> nrel_maintaining_organization: iso_organization;
```

Экземпляр `iso_organization` в базе уже есть — он записан как организация для Си и Си++. Вопрос для проверки: «Кто сопровождает Пролог?»

Опыт 2. Добавить новый экземпляр и связать с ним язык

Создать файл `concepts/instances/standards/iso_iec_13211_1_1995.scs` по образцу соседнего `concepts/instances/standards/iso_iec_9899_2018.scs`:

```
iso_iec_13211_1_1995
<- concept_language_standard;
<- concept_wit_entity;
=> nrel_main_idtf:
    [ISO/IEC 13211-1:1995]
    (* <- lang_ru;; *);
    [ISO/IEC 13211-1:1995]
    (* <- lang_en;; *);
```

```
=> nrel_note:
    [ISO/IEC 13211-1:1995 — международный стандарт языка программирования Пролог, часть 1: общее ядро.]
    (* <- lang_ru;; *);;
```

И в файл языка Пролог добавить строку:

```
=> nrel_language_standard: iso_iec_13211_1_1995;
```

Вопрос для проверки: «Какой стандарт у Пролога?» Файлов в базе станет на один больше: в опыте строка сборки показала `[1827/1827]` вместо `[1826/1826]`.

Опыт 3. Убрать строку — ответ сменится на «сведений нет»

Из файла `concepts/instances/languages/python_language.scs` убрать строку:

```
=> nrel_language_implementation: cpython_interpreter;
```

Вопрос для проверки: «Какая реализация у Python?»

После любой правки

```
docker compose restart problem-solver
docker compose restart py-sc-server
```

Дождаться сборки и задать вопрос. Строка в файле экземпляра должна заканчиваться точкой с запятой; две точки с запятой стоят только в самом конце описания.

После показа правку надо убрать и перезапустить ещё раз — иначе база перестанет совпадать с отчётом.

30. Откуда взялись файлы и как их пересоздать

Шестьдесят файлов одной формы руками не пишут. Здесь — чем они получены.

Три скрипта

Лежат в папке `tools` на компьютере, где работа делалась. В репозиторий не входят.

СКРИПТ	ЧТО СОЗДАЁТ	ИЗ ЧЕГО
`tools/gen_lr3_classify.py`	40 правил распознавания	из одного проверенного правила той же формы: меняются имена, классы, язык и четыре текста
`tools/gen_lr3_answer.py`	пять правил ответа	четыре — из правила об авторе, сделанного в ЛР2; правило «сведений нет» — из проверенного правила той же формы
`tools/gen_lr3_scs.py`	классы сообщений, классы фраз и фразы — 5 + 5 + 5 файлов	из таблицы с текстами внутри скрипта

Названия языков, формулировки и падежи записаны в `tools/kbinfo.py`. Значения отношений скрипты читают из файлов экземпляров.

Проверено: если удалить все 60 файлов ЛР3 и запустить три скрипта, получаются те же файлы байт в байт.

Когда это нужно

ЧТО ХОЧЕТСЯ ИЗМЕНИТЬ	КАК
Факт о языке	руками в файле экземпляра — раздел 29. Скрипты не нужны.
Текст фразы	руками в файле из `phrases/examples`. Меток `${…}` должно остаться две.
Формулировку вопроса или новый язык	поправить таблицы в `tools/kbinfo.py` и запустить генератор правил. Руками править сорок файлов `.gwf` не стоит.

Как это говорить на защите: правила получены из одного рабочего правила подстановкой имён, текстов и сущности. Все они одной формы — это видно, если открыть любые два.

ПРИЛОЖЕНИЯ

Приложение А. Все формулировки и ответы

40 сочетаний, 160 формулировок. Вопросы взяты из файлов правил, ответы — из фраз и названий в базе. Каждая формулировка задана в чате, ответ совпал с одним из двух указанных.

Тип вопроса × язык

ЯЗЫК	ЧТО В БАЗЕ	ФОРМУЛИРОВКИ, КОТОРЫЕ РАСПОЗНАЮТСЯ	ВОЗМОЖНЫЕ ОТВЕТЫ
Тип вопроса: автор · отношение `nrel_language_author`			
Python	`guido_van_rossum`	• Кто создал Python? • Кто автор Python? • Кто разработал Python? • Кто создал Python	• Автор языка программирования Python — Гвидо ван Россум. • Язык программирования Python создал Гвидо ван Россум.
Си	`dennis_ritchie`	• Кто создал Си? • Кто автор Си? • Кто разработал Си? • Кто создал Си	• Автор языка программирования Си — Деннис Ритчи. • Язык программирования Си создал Деннис Ритчи.
Си++	`bjarne_stroustrup`	• Кто создал Си++? • Кто автор Си++? • Кто разработал Си++? • Кто создал Си++	• Автор языка программирования Си++ — Бьёрн Страуструп. • Язык программирования Си++ создал Бьёрн Страуструп.
Java	`james_gosling`	• Кто создал Java? • Кто автор Java? • Кто разработал Java? • Кто создал Java	• Автор языка программирования Java — Джеймс Гослинг. • Язык программирования Java создал Джеймс Гослинг.

ЯЗЫК	ЧТО В БАЗЕ	ФОРМУЛИРОВКИ, КОТОРЫЕ РАСПОЗНАЮТСЯ	ВОЗМОЖНЫЕ ОТВЕТЫ
JavaScript	`brendan_eich`	• Кто создал JavaScript? • Кто автор JavaScript? • Кто разработал JavaScript? • Кто создал JavaScript	• Автор языка программирования JavaScript — Брендан Эйх. • Язык программирования JavaScript создал Брендан Эйх.
Haskell	`simon_peyton_jones`	• Кто создал Haskell? • Кто автор Haskell? • Кто разработал Haskell? • Кто создал Haskell	• Автор языка программирования Haskell — Саймон Пейтон Джонс. • Язык программирования Haskell создал Саймон Пейтон Джонс.
Пролог	`alain_colmerauer`	• Кто создал Пролог? • Кто автор Пролога? • Кто разработал Пролог? • Кто создал Пролог	• Автор языка программирования Пролог — Ален Колмероэ. • Язык программирования Пролог создал Ален Колмероэ.
язык ассемблера x86	нет	• Кто создал язык ассемблера x86? • Кто автор языка ассемблера x86? • Кто разработал язык ассемблера x86? • Кто создал язык ассемблера x86	• В базе знаний нет таких сведений об этом языке программирования. • Этого о данном языке программирования я пока не знаю.
Тип вопроса: год создания · отношение `nrel_creation_year`			
Python	`year_1991`	• В каком году создан Python? • Когда создан Python? • Когда появился Python? • В каком году создан Python	• Год создания языка программирования Python — 1991 год. • Python: год создания — 1991 год.
Си	`year_1972`	• В каком году создан Си? • Когда создан Си? • Когда появился Си? • В каком году создан Си	• Год создания языка программирования Си — 1972 год. • Си: год создания — 1972 год.
Си++	`year_1985`	• В каком году создан Си++? • Когда создан Си++? • Когда появился Си++? • В каком году создан Си++	• Год создания языка программирования Си++ — 1985 год. • Си++: год создания — 1985 год.
Java	`year_1995`	• В каком году создан Java? • Когда создан Java? • Когда появился Java? • В каком году создан Java	• Год создания языка программирования Java — 1995 год. • Java: год создания — 1995 год.

Язык	Что в базе	Формулировки, которые распознаются	Возможные ответы
JavaScript	`year_1995`	• В каком году создан JavaScript? • Когда создан JavaScript? • Когда появился JavaScript? • В каком году создан JavaScript	• Год создания языка программирования JavaScript — 1995 год. • JavaScript: год создания — 1995 год.
Haskell	`year_1990`	• В каком году создан Haskell? • Когда создан Haskell? • Когда появился Haskell? • В каком году создан Haskell	• Год создания языка программирования Haskell — 1990 год. • Haskell: год создания — 1990 год.
Пролог	`year_1972`	• В каком году создан Пролог? • Когда создан Пролог? • Когда появился Пролог? • В каком году создан Пролог	• Год создания языка программирования Пролог — 1972 год. • Пролог: год создания — 1972 год.
язык ассемблера x86	нет	• В каком году создан язык ассемблера x86? • Когда создан язык ассемблера x86? • Когда появился язык ассемблера x86? • В каком году создан язык ассемблера x86	• В базе знаний нет таких сведений об этом языке программирования. • Этого о данном языке программирования я пока не знаю.
Тип вопроса: реализация · отношение `nrel_language_implementation`			
Python	`cpython_interpreter`	• Какая реализация у Python? • Какая реализация используется для Python? • Чем реализован Python? • Какая реализация у Python	• Реализация языка программирования Python — CPython. • Для языка программирования Python используется реализация «CPython».
Си	`gcc_compiler`	• Какая реализация у Си? • Какая реализация используется для Си? • Чем реализован Си? • Какая реализация у Си	• Реализация языка программирования Си — GCC. • Для языка программирования Си используется реализация «GCC».
Си++	`gcc_compiler`	• Какая реализация у Си++? • Какая реализация используется для Си++? • Чем реализован Си++? • Какая реализация у Си++	• Реализация языка программирования Си++ — GCC. • Для языка программирования Си++ используется реализация «GCC».

ЯЗЫК	ЧТО В БАЗЕ	ФОРМУЛИРОВКИ, КОТОРЫЕ РАСПОЗНАЮТСЯ	ВОЗМОЖНЫЕ ОТВЕТЫ
Java	`java_virtual_machine`	• Какая реализация у Java? • Какая реализация используется для Java? • Чем реализован Java? • Какая реализация у Java	• Реализация языка программирования Java — виртуальная машина Java. • Для языка программирования Java используется реализация «виртуальная машина Java».
JavaScript	нет	• Какая реализация у JavaScript? • Какая реализация используется для JavaScript? • Чем реализован JavaScript? • Какая реализация у JavaScript	• В базе знаний нет таких сведений об этом языке программирования. • Этого о данном языке программирования я пока не знаю.
Haskell	нет	• Какая реализация у Haskell? • Какая реализация используется для Haskell? • Чем реализован Haskell? • Какая реализация у Haskell	• В базе знаний нет таких сведений об этом языке программирования. • Этого о данном языке программирования я пока не знаю.
Пролог	нет	• Какая реализация у Пролога? • Какая реализация используется для Пролога? • Чем реализован Пролог? • Какая реализация у Пролога	• В базе знаний нет таких сведений об этом языке программирования. • Этого о данном языке программирования я пока не знаю.
язык ассемблера x86	нет	• Какая реализация у языка ассемблера x86? • Какая реализация используется для языка ассемблера x86? • Чем реализован язык ассемблера x86? • Какая реализация у языка ассемблера x86	• В базе знаний нет таких сведений об этом языке программирования. • Этого о данном языке программирования я пока не знаю.
Тип вопроса: стандарт · отношение `nrel_language_standard`			
Python	нет	• Какой стандарт у Python? • Каким стандартом описан Python? • Какой стандарт действует для Python? • Какой стандарт у Python	• В базе знаний нет таких сведений об этом языке программирования. • Этого о данном языке программирования я пока не знаю.

ЯЗЫК	ЧТО В БАЗЕ	ФОРМУЛИРОВКИ, КОТОРЫЕ РАСПОЗНАЮТСЯ	ВОЗМОЖНЫЕ ОТВЕТЫ
Си	`iso_iec_9899_2018`	• Какой стандарт у Си? • Каким стандартом описан Си? • Какой стандарт действует для Си? • Какой стандарт у Си	• Стандарт языка программирования Си — ISO/IEC 9899:2018. • Язык программирования Си описан стандартом ISO/IEC 9899:2018.
Си++	`iso_iec_14882_2020`	• Какой стандарт у Си++? • Каким стандартом описан Си++? • Какой стандарт действует для Си++? • Какой стандарт у Си++	• Стандарт языка программирования Си++ — ISO/IEC 14882:2020. • Язык программирования Си++ описан стандартом ISO/IEC 14882:2020.
Java	нет	• Какой стандарт у Java? • Каким стандартом описан Java? • Какой стандарт действует для Java? • Какой стандарт у Java	• В базе знаний нет таких сведений об этом языке программирования. • Этого о данном языке программирования я пока не знаю.
JavaScript	`ecma_262`	• Какой стандарт у JavaScript? • Каким стандартом описан JavaScript? • Какой стандарт действует для JavaScript? • Какой стандарт у JavaScript	• Стандарт языка программирования JavaScript — ECMA-262. • Язык программирования JavaScript описан стандартом ECMA-262.
Haskell	нет	• Какой стандарт у Haskell? • Каким стандартом описан Haskell? • Какой стандарт действует для Haskell? • Какой стандарт у Haskell	• В базе знаний нет таких сведений об этом языке программирования. • Этого о данном языке программирования я пока не знаю.
Пролог	нет	• Какой стандарт у Пролога? • Каким стандартом описан Пролог? • Какой стандарт действует для Пролога? • Какой стандарт у Пролога	• В базе знаний нет таких сведений об этом языке программирования. • Этого о данном языке программирования я пока не знаю.

ЯЗЫК	ЧТО В БАЗЕ	ФОРМУЛИРОВКИ, КОТОРЫЕ РАСПОЗНАЮТСЯ	ВОЗМОЖНЫЕ ОТВЕТЫ
язык ассемблера x86	нет	• Какой стандарт у языка ассемблера x86? • Каким стандартом описан язык ассемблера x86? • Какой стандарт действует для языка ассемблера x86? • Какой стандарт у языка ассемблера x86	• В базе знаний нет таких сведений об этом языке программирования. • Этого о данном языке программирования я пока не знаю.
Тип вопроса: сопровождающая организация · отношение `nrel_maintaining_organization`			
Python	`python_software_foundation`	• Кто сопровождает Python? • Какая организация сопровождает Python? • Кто поддерживает Python? • Кто сопровождает Python	• Язык программирования Python сопровождает Python Software Foundation. • Сопровождением языка программирования Python занимается Python Software Foundation.
Си	`iso_organization`	• Кто сопровождает Си? • Какая организация сопровождает Си? • Кто поддерживает Си? • Кто сопровождает Си	• Язык программирования Си сопровождает Международная организация по стандартизации. • Сопровождением языка программирования Си занимается Международная организация по стандартизации.
Си++	`iso_organization`	• Кто сопровождает Си++? • Какая организация сопровождает Си++? • Кто поддерживает Си++? • Кто сопровождает Си++	• Язык программирования Си++ сопровождает Международная организация по стандартизации. • Сопровождением языка программирования Си++ занимается Международная организация по стандартизации.
Java	`oracle_corporation`	• Кто сопровождает Java? • Какая организация сопровождает Java? • Кто поддерживает Java? • Кто сопровождает Java	• Язык программирования Java сопровождает Oracle Corporation. • Сопровождением языка программирования Java занимается Oracle Corporation.
JavaScript	нет	• Кто сопровождает JavaScript? • Какая организация сопровождает JavaScript? • Кто поддерживает JavaScript? • Кто сопровождает JavaScript	• В базе знаний нет таких сведений об этом языке программирования. • Этого о данном языке программирования я пока не знаю.
Haskell	нет	• Кто сопровождает Haskell? • Какая организация сопровождает Haskell? • Кто поддерживает Haskell? • Кто сопровождает Haskell	• В базе знаний нет таких сведений об этом языке программирования. • Этого о данном языке программирования я пока не знаю.

ЯЗЫК	ЧТО В БАЗЕ	ФОРМУЛИРОВКИ, КОТОРЫЕ РАСПОЗНАЮТСЯ	ВОЗМОЖНЫЕ ОТВЕТЫ
Пролог	нет	• Кто сопровождает Пролог? • Какая организация сопровождает Пролог? • Кто поддерживает Пролог? • Кто сопровождает Пролог	• В базе знаний нет таких сведений об этом языке программирования. • Этого о данном языке программирования я пока не знаю.
язык ассемблера x86	нет	• Кто сопровождает язык ассемблера x86? • Какая организация сопровождает язык ассемблера x86? • Кто поддерживает язык ассемблера x86? • Кто сопровождает язык ассемблера x86	• В базе знаний нет таких сведений об этом языке программирования. • Этого о данном языке программирования я пока не знаю.

Приложение Б. Полный перечень файлов

Все файлы из папок `messages`, `logic_rules` и `phrases` с пометкой, в какой лабораторной сделаны. Файлов ЛР3 — 60, файлов ЛР2 — 64.

Пути от своей папки

Классы сообщений — `messages`

ЛР2

`concept_message_about_author.scs`

ЛР3

`concept_message_about_creation_year.scs`
`concept_message_about_implementation.scs`
`concept_message_about_language_property.scs`
`concept_message_about_maintainer.scs`
`concept_message_about_standard.scs`

Правила распознавания ЛР3 — `logic_rules/classification`

Тип «автор» — файлы с теми же именами были в ЛР2, в ЛР3 их содержимое заменено; файл для языка ассемблера x86 новый.

ЛР3

`lr_classify_author_of_python_lang_message.gwf`
`lr_classify_author_of_c_lang_message.gwf`
`lr_classify_author_of_cpp_lang_message.gwf`
`lr_classify_author_of_java_lang_message.gwf`
`lr_classify_author_of_javascript_lang_message.gwf`
`lr_classify_author_of_haskell_lang_message.gwf`
`lr_classify_author_of_prolog_lang_message.gwf`
`lr_classify_author_of_assembly_lang_x86_message.gwf`

Тип «год создания».

ЛР3

`lr_classify_creation_year_of_python_lang_message.gwf`
`lr_classify_creation_year_of_c_lang_message.gwf`

`lr_classify_creation_year_of_cpp_lang_message.gwf`
`lr_classify_creation_year_of_haskell_lang_message.gwf`
`lr_classify_creation_year_of_java_lang_message.gwf`
`lr_classify_creation_year_of_prolog_lang_message.gwf`
`lr_classify_creation_year_of_javascript_lang_message.gwf`
`lr_classify_creation_year_of_assembly_lang_x86_message.gwf`

Тип «реализация».

ЛР3

`lr_classify_implementation_of_python_lang_message.gwf`
`lr_classify_implementation_of_javascript_lang_message.gwf`
`lr_classify_implementation_of_c_lang_message.gwf`
`lr_classify_implementation_of_haskell_lang_message.gwf`
`lr_classify_implementation_of_cpp_lang_message.gwf`
`lr_classify_implementation_of_prolog_lang_message.gwf`
`lr_classify_implementation_of_java_lang_message.gwf`
`lr_classify_implementation_of_assembly_lang_x86_message.gwf`

Тип «стандарт».

ЛР3

`lr_classify_standard_of_python_lang_message.gwf`
`lr_classify_standard_of_javascript_lang_message.gwf`
`lr_classify_standard_of_c_lang_message.gwf`
`lr_classify_standard_of_haskell_lang_message.gwf`
`lr_classify_standard_of_cpp_lang_message.gwf`
`lr_classify_standard_of_prolog_lang_message.gwf`
`lr_classify_standard_of_java_lang_message.gwf`
`lr_classify_standard_of_assembly_lang_x86_message.gwf`

Тип «сопровождающая организация».

ЛР3

`lr_classify_maintainer_of_python_lang_message.gwf`
`lr_classify_maintainer_of_javascript_lang_message.gwf`
`lr_classify_maintainer_of_c_lang_message.gwf`
`lr_classify_maintainer_of_haskell_lang_message.gwf`
`lr_classify_maintainer_of_cpp_lang_message.gwf`
`lr_classify_maintainer_of_prolog_lang_message.gwf`
`lr_classify_maintainer_of_java_lang_message.gwf`
`lr_classify_maintainer_of_assembly_lang_x86_message.gwf`

Правила распознавания ЛР2 — та же папка

`logic_rules/classification/lr_classify_what_is_*.gwf` — 58 файлов, по одному на каждое понятие и экземпляр;
`logic_rules/classification/lr_classify_subdividing_of_*.gwf` — 2 файла.

Правила ответа — `logic_rules/answer`

ЛР2

`lr_answer_about_author.gwf`

ЛР3

`lr_answer_about_creation_year.gwf`
`lr_answer_about_standard.gwf`
`lr_answer_about_implementation.gwf`
`lr_answer_about_unknown_language_property.gwf`
`lr_answer_about_maintainer.gwf`

Классы фраз — `phrases`

ЛР2

`concept_phrase_about_author.scs`

ЛР3

`concept_phrase_about_creation_year.scs`
`concept_phrase_about_standard.scs`
`concept_phrase_about_implementation.scs`
`concept_phrase_about_unknown_language_property.scs`
`concept_phrase_about_maintainer.scs`

Сами фразы и образцы — `phrases/examples`

ЛР2

`concept_phrase_about_author.scs`

ЛР3

`concept_phrase_about_creation_year.scs`

`concept_phrase_about_implementation.scs`

`concept_phrase_about_maintainer.scs`

`concept_phrase_about_standard.scs`

`concept_phrase_about_unknown_language_property.scs`

Руководство собрано скриптом `tools/build_guide_lr3.py`: таблицы, листинги и числа берутся из файлов базы знаний и из записи проверочного прогона.